UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Boot process

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

From Firmware to Kernel

The Firmware Hand-off

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- BIOS/UEFI Initialization
  - Power-On Self-Test (POST): Hardware diagnostics and initialization
  - Detect and initialize CPU, RAM, I/O controllers
  - Scan storage devices (disks, USB, network) in configured boot order
  - Locate bootable disk with valid boot signature (0xAA55)
- Locating the Bootloader
  - **Legacy BIOS:** Reads Master Boot Record (MBR) from first 512 bytes of disk
  - **UEFI:** Loads kernel or GRUB (all stages together) from EFI System Partition (ESP) via EFI variables
- Control Transfer
  - CPU is placed in 16-bit real mode (BIOS) or 32/64-bit mode (UEFI)
  - Execution jumps to bootloader code (GRUB Stage 1)
  - Firmware services remain available for bootloader use
  - Note: After Stage 1 loads, firmware control is relinquished

GRUB Stages 1 + 1.5

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Stage 1

- The first 512 bytes, embedded in the MBR.
- Its only job is to locate and load Stage 1.5.
- Stage 1.5 is identified as a list of blocks to load (`blocklist`)
- Blockslist is created and stored when running `grub-install`

Stage 1.5

- Resides in the sectors immediately following the MBR (Post-MBR gap)
- Loads filesystem drivers (e.g., ext4, xfs).
- Allows GRUB to ”see” files instead of just disk blocks.

GRUB Stage 2 and Configuration

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Reading /boot/grub/grub.cfg

- This is GRUB's main configuration file.
- Contains menu entries for different kernels or operating systems.
- Defines default boot entry, timeout, and other settings.
- Generated automatically or manually edited.

The user menu and the passing of Kernel Parameters

- The user menu is the interface displayed for selecting boot options.
- Kernel parameters are options passed to the Linux kernel at boot time.
- Examples: `root=UUID=...` (specifies the root filesystem), `ro` (mount root read-only), `init=...` (specifies the init system).
- Allows customization of kernel behavior without recompiling.

The Kernel image (vmlinuz)

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Boot Header: The initial part of the kernel image that provides essential information for the bootloader, including the kernel's entry point, architecture specifics, and setup code
- Decompression Routine: A small program embedded in the kernel image that decompresses the compressed payload into RAM
- Compressed Kernel Payload: The core kernel code and data structures, compressed using algorithms like gzip or LZMA to reduce the image size

Kernel boot sequence

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Entry Point Execution: The bootloader jumps to the kernel's entry point, initiating the boot sequence
- Payload Decompression: The embedded decompression routine unpacks the compressed kernel into a temporary high-memory area memory
- Memory reclaiming: The compressed image is no longer useful and its memory is considered available
- Initialization Phase: The kernel begins initializing hardware, memory management, and essential subsystems. Starting point is `startup_64()` on x86 architecture

CPU and Memory Initialization

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Transitioning from Real Mode to Protected Mode (and finally Long Mode for 64-bit)

- Real Mode: Initial 16-bit mode after BIOS, with segmented addressing and 1MB physical address space
- Protected Mode: 32-bit mode enabling segmentation, protection rings, and up to 4GB address space via GDT
- Transition to Protected Mode: Load GDT with `lgdt`, set `PE` (Protection Enable) bit in `CR0` register
- Long Mode: 64-bit extension (AMD64), requires paging for full 64-bit addressing
- Transition to Long Mode: Set up initial page tables, enable `LME` (Long Mode Enable) in `EFER` (Extended Feature Enable Register), set `PG` (Paging) bit in `CR0` for paging

GDT and Paging Setup

Setting up the GDT (Global Descriptor Table) and the initial Page Tables for virtual memory management

- Global Descriptor Table (GDT): Defines memory segments, including code and data descriptors for flat memory model
- Initial GDT Setup: Kernel sets up basic descriptors for kernel code and data segments
- Page Tables: Hierarchical structure for 4-level paging in x86-64 architecture
- Initial Page Tables: Identity mapping of kernel image and data, enabling virtual memory translation

Initramfs

- A “Chicken and Egg” Problem
- The kernel needs to mount the root disk (/)
- The disk might be on a RAID array, an encrypted partition (LUKS), or a network drive (NFS)
- The Dilemma: The drivers to read these are on the disk we haven’t mounted yet
- Solution: a minimal root filesystem available for kernel operations
- Introducing Initramfs

Anatomy of initramfs

- It is a CPIO archive compressed with gzip/xz
- Content: A miniature root filesystem with the following structure
  - `/bin`: essential binaries
  - `/lib`: shared libraries
  - `/init`: the initialization script
  - `/dev`: device nodes
  - `/proc`: proc filesystem mount point
  - `/sys`: sys filesystem mount point
  - `/etc`: configuration files

The init Script in RAM

- The kernel mounts the initramfs as a temporary root filesystem
- The script in `/init` is the first user-space process executed from RAM
- It mounts pseudo-filesystems like `/proc`, `/sys` and `/dev`
- It starts hardware discovery tools (`udev`) and loads needed drivers
- It assembles storage stacks: partitions, LVM, RAID, and handles disk decryption if needed
- Finally it locates the real root filesystem, switches to it, and hands off control to the real `init`

Finding the *True* Root

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The kernel does not know the root filesystem until boot time
- GRUB passes a `root=` parameter on the kernel command line to identify the root device
- Device names like `/dev/sda1` are not stable across reboots
  - The Linux kernel assigns names like `sda`, `sdb`, or `nvme0n1` based on the order in which the hardware is discovered
  - Plugging a drive into a different USB or SATA port changes its name
- UUID or label (filesystem metadata) is used instead to identify the root partition reliably
  - Disks can be identified by UUID in the `/dev/disk/by-uuid/` directory
  - Disks can be identified by label in the `/dev/disk/by-label/` directory
- The initramfs must resolve the real root device before switching from RAM to the actual filesystem

The pivot_root syscall

- Syscall that switch the current root partition
- `pivot_root` requires manual setup
  - The new root must be a mount point
  - The old root is pivoted to a subdirectory without executing a new program
- `pivot_root` can be used in various contexts (e.g., containers), whereas `switch_root` is primarily for system boot
- Cannot be used to exit the initial rootfs, must use `chroot` syscall

The switch_root utility

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Userspace utility (part of `util-linux`) called by the init script in the initramfs once the real root filesystem is accessible
- Takes two arguments
  - The path to the new root directory
  - The path to the init program
- Moves the current root filesystem (initramfs) to a subdirectory of the new root (e.g., `/initramfs`)
- Changes the root directory to the new filesystem
- Removes the previous root partition, freeing the memory of the initramfs
- Executes the `init` program, replacing the current process

Executing the Userspace Init

- The kernel looks for `/sbin/init` on the real root disk
- It uses `execve()` to replace its own initialization code with this process
- Performs hand-off
  - The kernel enters ”Passive Mode” (handling interrupts and syscalls)
  - PID 1 takes over the logical orchestration of the OS

Critical aspects

- Safety: initramfs is a safety net that allows Linux to boot on thousands of different hardware configurations using the same generic kernel
- Abstraction: the software “hides” the complexity of the hardware (RAID/Encryption) from the final OS environment.
- Efficiency: the memory used by the bootloader and the initramfs is reclaimed by the kernel once the boot is successful. Nothing is wasted.

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

System V Init (`SysV`)

Philosophy

- The System V Init Model
  - A.K.A. `SysV`
- Key Concept: “Everything is a script”
- Follows the Unix Philosophy
  - Small, modular shell scripts
  - Each script does one thing well
- PID 1 Role: init acts as a manager that calls external scripts rather than handling service logic internally.

The Logic of Runlevels

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- A runlevel is a software configuration of the system that allows only a selected group of processes to exist
- Runlevels
  - 0: Halt (Shut down)
  - 1 (S): Single-user mode (Maintenance/Root shell)
  - 2: Default runlevel on Debian. No-NFS mode on Red-Hat
  - 3: Multi-user mode (Command Line Interface)
  - 4: User-defined
  - 5: Multi-user mode (Graphical User Interface)
  - 6: Reboot.
- Boot parameters are typically used by the kernel
- Boot parameters not used by the kernel are passed to init
  - Can specify the actual init command using the `init=` kernel parameter
  - Can specify runlevel at boot time passing a parameter to init

The `telinit` Command

- Purpose: Used to communicate with the `init` process (PID 1)
- Primary Function: Manually change the current system runlevel
- Syntax: `telinit RUNLEVEL|OPTION`
  - Option `q` or `Q`: Tells `init` to parse `/etc/inittab` again

Examples:

```
# Switch to multi-user mode (Runlevel 3)
telinit 3
# Reload /etc/inittab after making changes
telinit q
# Reboot the system
telinit 6
```

`/etc/inittab` file

- The configuration file for init
- Content Breakdown
  - Each entry follows the format: `id:runlevels:action:process`
  - `id`: A unique identifier (1-4 characters)
  - `runlevels`: Runlevels where this entry applies (e.g., 12345)
  - `action`: Action to take (e.g., `initdefault`, `sysinit`, `wait`, `respawn`, `ctrlaltdel`)
  - `process`: The command or script to execute
  - Examples:
    - Default runlevel: `id:3:initdefault:`
    - Respawn getty: `1:2345:respawn:/sbin/getty tty1`
- Execution: init parses this file to determine the initial system state

Example `/etc/inittab` file

```
# /etc/inittab - simple example
# set the default runlevel to 3
id:3:initdefault:
# Execute /etc/init.d/rcS at startup
si::sysinit:/etc/init.d/rcS
# Execute login teminal using getty command #runlevel 2 3 4 5
1:2345:respawn:/sbin/getty 38400 tty1
# Reboot when Ctrl+Alt+Del is pressed
c:12345:ctrlaltdel:/sbin/shutdown -t3 -r now
```

`/etc/init.d/` script repository

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Centralization: All “master” scripts reside here
- Script Structure: Standard Bash scripts that accept arguments. Possible rguments include
  - `start`: start the service
  - `stop`: stop the service
  - `restart`: restart the service
  - `reload`: reload configuration (without restarting)
  - `status`: print the service status
- Standardization: The LSB (Linux Standard Base) headers used to define script metadata

Example server

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <time.h>
#define SOCKET_PATH "/tmp/mytime"
#define BUFFER_SIZE 32
int main() {
  int server_fd, client_fd;
  struct sockaddr_un addr;
  char buffer[BUFFER_SIZE];
  // socket setup
  if ((server_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == -1) {
    perror("socket error"); exit(EXIT_FAILURE);}
  memset(&addr, 0, sizeof(struct sockaddr_un));
  addr.sun_family = AF_UNIX;
  strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);
  unlink(SOCKET_PATH);
```

Example server

```
  // binding
  if (bind(server_fd, (struct sockaddr *)&addr, sizeof(struct sockaddr_un)) == -1) {
    perror("bind error"); exit(EXIT_FAILURE);}
  if (listen(server_fd, 5) == -1) {perror("listen error"); exit(EXIT_FAILURE);}
  // main loop
  while (1) {
    if ((client_fd = accept(server_fd, NULL, NULL)) == -1) {
      perror("accept error"); continue; }
      time_t now = time(NULL); // get time
      snprintf(buffer, BUFFER_SIZE, "%ld\n", (long)now);
      write(client_fd, buffer, strlen(buffer));
      close(client_fd);
  }
  close(server_fd);
  return 0;
}
```

Testing the server

- To run the daemon simply compile and invoke it
  - `cc -o timed timed.c`
  - `./timed`
  - Check if the socket appeared
- To test using a client the daemon
  - `nc -U /tmp/mytime` (not available for GNU netcat)
  - `socat - UNIX-CONNECT:/tmp/mytime`
- To install the service in `/usr/local/bin`
  - `sudo install -m 755 -o root -g root ./timed /usr/local/bin/`

/etc/init.d/mytime script

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```
#!/bin/sh
### BEGIN INIT INFO
# Provides: timed
# Required-Start: $local_fs $network
# Required-Stop: $local_fs $network
# Default-Start: 2 3 4 5
# Default-Stop: 0 1 6
# Short-Description: Unix Socket Time Daemon
### END INIT INFO
PATH=/sbin:/usr/sbin:/bin:/usr/bin
DESC="Unix Socket Time Daemon"
NAME=timed
DAEMON=/usr/local/bin/timed # Path to your compiled C binary
PIDFILE=/var/run/$NAME.pid
SCRIPTNAME=/etc/init.d/$NAME
# Exit if the binary is not installed
[ -x "$DAEMON" ] || exit 0
. /lib/lsb/init-functions
```

/etc/init.d/mytime script

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```
case "$1" in
  start)
    start-stop-daemon --start --quiet --pidfile $PIDFILE --make-pidfile --background
         --exec $DAEMON
    ;;
  stop)
    start-stop-daemon --stop --quiet --retry=TERM/30/KILL/5 --pidfile $PIDFILE
    RETVAL="$?"
    [ "$RETVAL" = 2 ] && return 2
    # Clean up the PID file and the socket file
    rm -f $PIDFILE
    rm -f /tmp/mytime
    ;;
  *)
    echo "Usage: $SCRIPTNAME {start|stop|status|restart}" >&2
    exit 3
    ;;
esac
exit 0
```

`/etc/rcX.d/` state transitions

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The Directory Structure: For every runlevel *X*, there is a directory `/etc/rcX.d/`
- The Symlink Model: These directories don't contain real files, only symbolic links pointing back to `/etc/init.d/`
- Symlinks allows one script to be used in multiple runlevels without duplication.

Link names

- Naming Convention
  - S (Start): Services to be launched when entering the runlevel
  - K (Kill): Services to be terminated when entering the runlevel
- Numeric Priority: The two digits following S/K (e.g., `S10network` vs `S80apache`) define the sequential order
- Logic: `S10network` must finish before `S80apache` begins

Execution Flow: The Transition

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- User requests change (e.g., `telinit 5`)
- init identifies the target `/etc/rc5.d/`
- Phase 1: Run all `K*` scripts in numeric order
- Phase 2: Run all `S*` scripts in numeric order
- Result: The system reaches the new stable state.

The limits of SysV

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- System V is now a legacy code
- Limits of the approach (discussed in the next slides)
  - Sequential bottleneck
  - Shell overhead
  - Static nature

The Sequential Bottleneck

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Blocking Execution: Scripts run one after another in a single “thread” of execution
- The Network Timeout Problem: If a script (like DHCP) waits for a 60-second timeout, the entire boot process hangs at that line
- Result: Slow boot times on modern multi-core hardware.

Shell Overhead

- Performance Cost: Each script requires forking a new shell (/bin/sh)
- Resource Waste: In a system with 80+ services, the overhead of context switching and process creation for shell execution becomes significant.

Static Nature vs. Hot-Plugging

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Hardware Assumptions: SysV assumes hardware is static and detected at boot
- The Modern Challenge: How does SysV react if a USB network adapter is plugged in after the network script has already run?
- Manual Intervention: Requires the user to manually restart scripts (service network restart)
- Need for external tools that wraps scripts (e.g., `udev`, `Network Manager`)

Fragile Process Tracking

- The PID File Problem: SysV tracks services by writing a PID to a file (e.g., `/var/run/apache.pid`)
  - Need code to manage PID files
  - Use of external wrappers (e.g., `start-stop-daemon`)
- Orphans: If a service crashes or forks incorrectly, the “child” processes are left running (zombies/orphans),
- init has no way to track them because they aren’t in a containerized environment

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

systemd

The Paradigm Shift

- From Imperative to Declarative: Moving from ”How to start” (Bash scripts) to ”What state is desired” (Unit files)
  - Imperative: Scripts specify step-by-step instructions on how to start services, requiring manual sequencing and error handling.
  - Declarative: Unit files describe the desired end state, allowing systemd to automatically determine and execute the necessary steps.
- The ”One vs. Many” Philosophy: systemd is not just an init system; it is a system management suite
  - Unified suite: Combines init functionality with logging (`journald`), device management (`udev`), session handling (`logind`), and more.
  - Holistic approach: Aims to manage the entire system lifecycle, reducing the need for separate tools and improving integration.

Key Innovation: Parallelism

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The SysV Serial Problem: Traditional init is limited by the slowest script
  - Boot scripts execute one after another in a fixed sequence.
  - A single slow or blocked init script stalls the entire boot.
  - Dependencies are encoded by order, not by explicit relationships.
- The systemd Solution: Starting services in parallel by resolving dependencies before the services are even fully ready
  - systemd builds a dependency graph from unit files.
  - Independent services can start simultaneously instead of waiting in line.
  - The daemon can launch units before all targets are reached, reducing idle time.
- The Hardware Reality: Modern systems have multiple CPU cores; systemd utilizes them
  - Parallel startup allows multiple CPU cores and devices to work together.
  - I/O and CPU work overlap, improving utilization during boot.
  - The result is a faster boot on multi-core systems with concurrent workloads.

Socket Activation

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Systemd creates the socket (Network or Unix) on behalf of the service
- The Flow
  - systemd can listen on socket (e.g., `/tmp/mytime`)
  - A client connects
  - systemd ”wakes up” the daemon and passes the connection
  - Like Wietse Venema TCP wrapper `tcpd`
- Benefit: Services can start simultaneously because the ”socket” is available even if the daemon isn’t ready yet.

D-Bus Activation

- D-Bus overview
  - System-wide communcation bus: defines enpoints such as `org.freedesktop.NetworkManager`, methods that can be invoked and data representation
  - Two types of bus session: one for system and one for each user session
  - Subsystems using D-Bus include `NetworkManager`, `CUPS`, and `GNOME` for both sessions and settings management
- D-Bus and systemd
  - IPC-driven startup: Services are launched only when another process tries to communicate with them via the System Bus
  - Efficiency: Reduces memory footprint by not running idle daemons until they are explicitly called

Introduction to Units

- Unit Files: The building blocks of systemd (.service, .socket, .device, .mount)
- Structure: Simple INI-style configuration files
- Validation: Unlike shell scripts, unit files can be parsed and validated for syntax before execution

Type of unit file

File type	Use	Replacing
`.service`	Programs/Daemons	Init scripts (`/etc/init.d/`)
`.target`	System States	Runlevels
`.timer`	Schedules	Cron jobs
`.mount`	Disks/Folders	`/etc/fstab` entries
`.socket`	Network/IPC	`inetd` / `xinetd`

Units vs. Runlevels

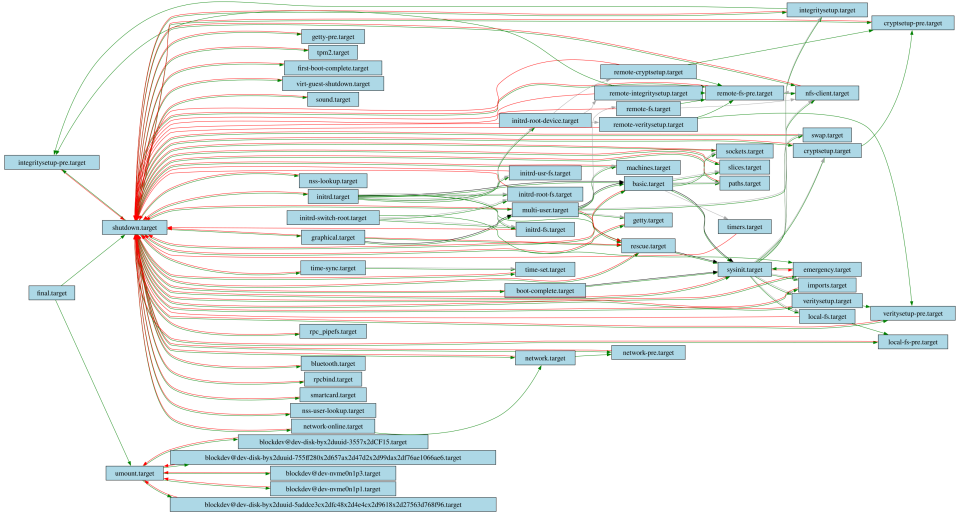
- Readability: Replacing abstract numbers (0-6) with human-readable Targets (e.g., `poweroff`)
- Flexibility: Targets can be combined and nested, whereas you can only be in one runlevel at a time.

Essential Targets

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Runlevel-equivalent targets
  - `poweroff.target` (Runlevel 0)
  - `rescue.target` (Runlevel 1)
  - `multi-user.target` (Runlevel 3)
  - `graphical.target` (Runlevel 5)
  - `reboot.target` (Runlevel 6)
  - `default.target`: default working setup (e.g., a link to `multi-user.target` or `graphical.target`)
- Capability-related targets
  - `network.target`: network management software has started
  - `basic.target`: the system is considered *minimally functional*
  - `local-fs.target`: all local filesystems listed in `/etc/fstab` are successfully mounted
- To change the system status: `systemctl rescue.target`

Target graph



Systemd unit files

- Declarative Syntax: Units use an INI-style format (`[Section]`) composed of Key=Value pairs
- The common sections
  - [Unit]: Metadata and relationship logic. It defines how this unit relates to the rest of the system graph
  - [Install]: The “hook-up” logic. It defines when this unit should be activated (e.g., during which Target)
- Unit-specific sections
  - [Service]: The execution logic of a `.service` unit. It defines what to run and how to manage the process
  - [Timer]: Definition of the trigger in `.times` unit. It defines when a unit should be activated
  - [Mount]: Definition of a mount point in `.mount` unit. It defines what to mount, how and where
  - [Socket]: Definition of an IPC endpount in `.socket` unit. It defines where to listen and what to do

systemd service

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```
[Unit]
Description=Unix Socket Time Daemon
After=network.target

[Service]
# Path to compiled C binary -- must be absolute path
ExecStart=/usr/local/bin/timed
# Automatically restart if the process crashes
Restart=on-failure
# systemd tracks the process via Cgroups, so it doesn't need a PID file
Type=simple
# User
User=nobody
# Logging
StandardOutput=journal

[Install]
# This defines which "Runlevel" equivalent to use
WantedBy=multi-user.target
```

Dependency Graphs (Wants / Requires)

- The description of the DAG can model several aspects
- Dependency description
  - `Wants=`: A weak dependency. If the dependent service fails, the main service still starts
  - `Requires=`: A strong dependency. If the dependent service fails, the main service fails too
- Ordering vs. Dependency: `After=`, `Before=` only dictates when to start, not if it must start

Keyword	Type	Logic
`Requires`	Requirement	Mandatory. If the dependency dies, this unit dies
`Wants`	Requirement	Optional. Recommended for most services
`After`	Ordering	Wait for the other unit to finish initializing
`Before`	Ordering	Make the other unit wait for us
`Conflicts`	Negative	There can be only one. Shuts down the other unit

Service installation

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Installation of a service
  `systemctl enable timed.service`
- systemd looks at the `[Install]` section in the `.service` file
- If a `WantedBy=<target>` statement is found (in our case `multi-user.target`)
  - Go to directory `/etc/systemd/system/multi-user.target.wants/`
  - Create symlink
    `timed.service` → `/etc/systemd/system/timed.service`
- At next boot
  - When systemd reaches the `multi-user.target`
  - Looks inside `.../multi-user.target.wants` directory
  - Activates every service in it (including `timed.service`)

Non-Linear Boot Sequence

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Systemd models the boot process as a Directed Acyclic Graph (DAG).
  - Nodes: Represent ”Units” (services, mount points, sockets).
  - Edges: Represent dependencies (`Requires`, `Wants`) and ordering (`After`, `Before`).
- Parallelization Strategy:
  - Independent branches of the DAG are executed simultaneously, reducing total boot time.
  - Leverages multi-core architectures better than sequential `rc` scripts.
- Optimization & Pathfinding:
  - Target Units: Acts as synchronization points (e.g., `multi-user.target`).
  - Shortest Path: systemd calculates the minimal set of units required to reach a specific target.
  - Cycle Detection: During parsing, systemd must verify the graph is acyclic; circular dependencies (A requires B, B requires A) are flagged as errors.
- Visualization: Tools like `systemd-analyze dot` can export the runtime graph into Graphviz format for topological analysis.

Cgroups: Process Tracking

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The Orphan Problem
  - In SysVinit, `init` tracks services only by their PID (Process ID)
  - "Double-forking" (creating a child of a daemon that is then killed) allows the child to escape its parent's session
  - The PID file is not useful to find child of the original daemon
  - Result: If a service dies or is reloaded, "zombie" or "orphan" subprocesses remain, consuming resources and leaking memory
- Control Groups Cgroups at the kernel level
  - systemd places every service into its own dedicated Control Group.
  - Unlike PIDs, Cgroup membership is inherited and cannot be "escaped" by forking
  - Hierarchical Organization: Processes are organized into a tree structure (`/sys/fs/cgroup/`)
  - Live view of systemd status: `systemd-cgls`

Cgroups: Resource Management

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Resource Management & Isolation:
  - Accounting: Real-time tracking of CPU, RAM, and I/O usage per service.
  - Throttling: Hard limits can be set to prevent a single daemon from starving the system.
- Reliable Termination:
  - `systemctl stop` sends signals to the *entire group* simultaneously.
  - Ensures "Clean Room" exits: No subprocess can survive the termination of the service unit.

Resource Constraints: QoS & Isolation

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Resource Controllers: Kernel subsystems that enforce limits on Cgroups.
  - CPU: Uses the Completely Fair Scheduler (CFS) bandwidth control.
  - Memory: Monitors resident set size (RSS) and address space limits.
  - Block I/O: Weights or limits disk read/write operations (IOPS/bps).
- Exceeding the resource quota activates the kernel Out-Of-Memory (OOM) killer
- Declarative Constraints in Unit Files

```
[Service]
# Limit to 50% of a single CPU core
CPUQuota=50%
# Hard limit; OOM Killer triggers if exceeded
MemoryMax=2G
# Prevent "Fork Bombs" by limiting max processes
TasksMax=100
```

- Performance Isolation
  - Prevents *Noisy Neighbor* syndrome in multi-tenant environments.
  - Guarantees that critical system targets (e.g., `ssh.service`) remain responsive even under high system load.

Unified Logging with `systemd-journald`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Structured Logging: Shift from flat text files (`syslog`) to a binary, indexed format
  - Speed: Indexed logs allow near-instant searching by time or service.
  - Integrity: Binary format is harder to tamper with than plain text
- Automatic Metadata Enrichment:
  - Every entry is tagged with kernel-verified metadata: `PID`, `UID`, `GID`, `Cgroup`, and `SELinux context`
  - This eliminates the “Who logged this?” ambiguity found in traditional logs.
- Centralization: Captures `stdout` and `stderr` from all processes.
  - No more manual configuration of log paths for individual daemons.

```
# Filter by Unit (Service)
journalctl -u my-daemon.service
# View kernel messages (dmesg style)
journalctl -k
# Show only errors since last boot
journalctl -p err -b
```

Service Monitoring and Self-Healing

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Active Health Checking (Watchdogs)
  - The service sends keep-alive pings (heartbeats) to systemd via `sd_notify()`
  - If the service hangs (deadlock) and stops pinging, systemd detects the failure even if the process is still running
- Declarative Restart Policies
  - `Restart=on-failure`: Restarts only if the exit code is non-zero or it timed out.
  - `Restart=always`: Mimics SysV `respawn`, but with intelligent throttling.
  - Backoff Strategy: `RestartSec=5s` prevents "rapid-fire" restarts that could saturate the CPU during a persistent crash loop.

Comparison: SysV `respawn` vs. systemd

Feature	SysVinit (respawn)	systemd
Detection	Process Exit Only	Exit + Watchdog Hangs
Granularity	System-wide limit	Per-service config
Safety	Hard to throttle	Exponential backoff options

Performance Profiling: `systemd-analyze`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Empirical Boot Analysis: Tools to measure the Time to Userspace
  - Distinguishes between Kernel time, initrd time, and Userspace time
- Bottleneck Identification
  - `systemd-analyze blame`: Ranks units by their initialization time
  - `systemd-analyze critical-chain`: Displays the dependency chain responsible for the slowest path to a target (Critical Path Method).
- Visualizing Concurrency
  - `systemd-analyze plot > boot.svg`: Renders a detailed SVG timeline.
  - Students can see exactly which services started in parallel and which were blocked by dependencies.

```
# Print total time spent in kernel vs userspace
systemd-analyze
# Find the services slowing down the boot
systemd-analyze blame | head -n 5
# Trace the critical path for the graphical interface
systemd-analyze critical-chain graphical.target
```

Wrap up: benefits of systemd

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Architectural Consistency
  - Standardizes administration across Fedora, Debian, Ubuntu, and Arch
  - Moves from imperative Shell scripts to declarative Unit files
- Performance Through Parallelism
  - Uses DAG-based scheduling to saturate multi-core CPUs
  - Aggressive socket and D-Bus activation allows services to start only when needed
- Kernel-Level Robustness
  - Cgroups: Zero-leakage process tracking; solves the “Double Fork” escape
  - Predictability: Strict dependency handling prevents race conditions during boot
- Integrated Observability
  - Unified, tamper-resistant logging with `journald`
  - Built-in profiling tools (`systemd-analyze`) for performance tuning

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Hands-on

Overview

- Tracing the Boot: Using `dmesg` to view kernel logs
- Analyzing Performance using `systemd-analyze blame` to find slow-starting services
- Analyzing critical path using `systemd-analyze critical-chain`
- Service Creation: Writing a simple systemd unit file to manage a custom daemon
- Show graph of services
  - General graph `systemd-analyze dot | dot -Tsvg > dependencies.svg`
  - Per-service graph `systemd-analyze dot timed.service | dot -Tsvg > timed_deps.svg`